Beyond Exact Copies: Detecting Edited Copies of AI Agent Skills

Bhavesh Maurya
bmaurya@stevens.edu
Stevens Inst. of Tech.

Abhishek Thakkar
athakkar1@stevens.edu
Stevens Inst. of Tech.

Mahima Jain
mjain19@stevens.edu
Stevens Inst. of Tech.

Pratiksha Vilas Pawar
ppawar6@stevens.edu
Stevens Inst. of Tech.

Abstract

Developers reuse agent skills, the `SKILL.md` instruction files that coding agents load on demand, by copying them between repositories and frequently modifying them in the process. In the absence of a package manager, nothing records where a modified copy came from. Current work on reuse links copies through registry names, forks, or identical versions, while clone detection based on similarity alone cannot distinguish a modified copy from another skill generated from the same template. We annotate 400 real skill pairs from the GitSkills dataset as derived variants, template siblings, or unrelated (Cohen's $\kappa = 0.95$) and train a classifier using lexical, boilerplate-aware, metadata, code, structural, and repository features, with boilerplate lines counted over all 1.59 million in-scope skills of the dataset. It classifies derived variants with an F1 of 0.94 (95% CI 0.89–0.97), against 0.74 for a SkillReuse-style baseline, and does not classify any of the 159 highly similar non-derived pairs as derived. Applied to 9.0 million candidate pairs, it predicts 4.1 million derivation links; a blinded manual check of 80 randomly drawn links confirms 79 (precision 0.99, 95% CI 0.93–1.00). Counting reuse by derivation family instead of by exact file hash raises the Gini coefficient of reuse from 0.49 to 0.61–0.63 and the share of files in the top 1% of units from 27% to 40–42%, so reuse is much more concentrated than exact copies suggest.

1. Introduction

Agent-based assistants like Claude Code have been integrated into the daily routine of software developers, who use them to edit files, run commands, and check code changes on their behalf. In order for an agent to do a job as specified by the team's procedure, developers define the procedure as an *agent skill*. A skill is a directory containing a `SKILL.md` file, which consists of YAML front matter including a `name` and a `description`, followed by instructions in Markdown; scripts and reference documents may be included in the directory as well [3, 11]. At first, the agent reads just the descriptions of the skills, and it loads the detailed instructions of a skill only when the task fits its description. The format was introduced as an open standard by Anthropic in October 2025, and just nine months later millions of `SKILL.md` files were stored in public GitHub repositories [11].

The libraries included in an application through npm or PyPI are recorded in the manifest file, pinned to a specific version, and installed with the help of the package manager. Skills do not have such an installation procedure. Developers simply copy the directory with the skill into their code base, with any edits performed along the way, without leaving any trace of the origin of the skill. The GitSkills dataset shows how common this is: of the 3,797,117 `SKILL.md` files found in 282,200 repositories, 50.5% are exact binary copies of another file in the dataset [11]. Edited copies are not counted in this statistic, since any edit changes the content hash.

The following definitions are used throughout the paper. An *exact copy* has the same content hash as another file. A *derived variant* is a skill created by copying another skill and modifying it through renaming, trimming, extending, rephrasing, or translation; two skills copied from the same source are derived variants of each other. A *template sibling* has only a generic structure in common with another skill, such as section names, general instructions, or the style of a catalog, while the rest of its instructions were written independently. A group of skills connected by derivation forms a *skill family*, and the derivation links within a family form its *lineage*. We call reuse *hidden* when two skills are derived from one another but share neither a content hash nor a front-matter name, so that neither exact matching nor name matching can connect them. These notions are borrowed from clone detection in software engineering, where clones range from exact copies (Type-1) to copies with renamed identifiers (Type-2) and copies with added, removed, or changed statements (Type-3) [16].

Lineage is important because an agent executes a skill instead of just showing it to a reader. A human who reads an out-of-date README file may notice the issue and work around it, but an agent simply follows the out-of-date instructions of a skill [12]. Skills can even be dangerous. Audits of publicly available skills have found both vulnerable and malicious skills [14], and in January 2026, one campaign alone introduced more than 1,200 malicious skills to the OpenClaw skill marketplace [20]. After a flawed skill is copied and altered, finding its descendants becomes difficult.

A concrete case from our data shows the problem. One skill in GitSkills is a verbatim copy of a ClickHouse database skill, published under a different name, with an added section that sets up a reverse shell (pair V103 of our validation sample, taken from a benchmark of malicious skills). The two files share neither a content hash nor a name, so neither exact nor name matching can connect the malicious copy to its benign source. Our classifier links them, and both annotators confirmed the link.

To address these challenges, we investigate three research questions:

RQ1. *How accurately does our approach recognise derived skill variants, compared with exact hashing, similarity thresholds, and SkillReuse?*

RQ2. *Which signals contribute most to separating derived skill variants from template siblings and unrelated skills?*

RQ3. *How much skill reuse is hidden from exact and name matching, and how does counting it change the picture of reuse?*

2. Related Work

2.1. Reuse and adaptation of agent skills

An early analysis of skills as engineered artifacts is due to Gao et al. [12]. They gathered 18,463 skills from the skills.sh registry and 23,199 skills from 5,876 GitHub repositories. To link a repository

skill to a registry skill, they first performed an exact match on the YAML `name`, then used longest-common-subsequence similarity to pick one of the candidates with the same name, and dropped all pairs with less than 0.1 similarity. This yielded 3,709 reuse links. Their results revealed that reuse is often near-exact: 53% of the reused skills were never changed once adopted. Of the reused skills that were never changed locally, 40.2% were taken from a registry whose source later changed, while the copies stayed the same. Yet their method only detects copies whose source is known and available on skills.sh and whose YAML name was not changed. They point out that renamed and modified skills are not detected, and that the whole process assumes that reuse goes from the registry to the repositories. In their dataset, 76% of the collected repository skills are not linked, and some of them may well be reuse that was not detected.

Wu et al. [19] investigated how published skills are adapted by examining the public forks of six prominent skill repositories. On the basis of 1,126 adaptation instances, they provide a taxonomy of 46 adaptation patterns organised in 13 families and report that 18.6% of the adaptations add security-sensitive content to the instructions. They also recommend the development of tools that detect inconsistent adaptations. As their results are derived from the GitHub fork relationship, they only apply to copies made by forking those six repositories; when a skill is pasted into an unrelated repository, no fork relationship is left.

2.2. Reconstructing how skills spread

Seddik [18] reconstructs the propagation of skills in the same GitSkills release as ours. The study clones the default branch of every repository in the dataset, collects the git history of each `SKILL.md` file, and constructs a dated, directed copy network of 2,193,119 skill adoptions. A few repositories turn out to be the origin of almost all copies, and GitHub stars do not help to detect them. Copies also rarely stay synchronised with their origins: only 11.1% of the changes applied to a set of identical copies reach all copies in the set, and only 20.9% of copies adopt a later edit of their origin. The network is built from exact content. Each unique content corresponds to one skill, and the versions of a skill are connected to each other through the edits recorded by git. In our interpretation of the method, copies edited before their first commit share no version with their origin and are therefore not connected to it; the same applies to renamed and translated copies. The study thus shows where an exact copy came from and whether fixes propagate, but it leaves edited copies untouched.

2.3. Clone detection for agent skills

Of all skill reuse detectors, SkillReuse [20] is the most similar to our work. It encodes the YAML metadata, the prose, and the code of a skill as separate channels, and uses the similarities between these channels, together with a bare TF-IDF similarity and structural features, as input to a logistic regression that reaches an F1 of 0.939 on SkillReuse-Bench, a dataset of 300 labelled skill pairs. When run on 137,470 skills scraped from SkillsMP, it places 66.8% of skills in at least one clone pair, predominantly across authors. The authors explicitly point out two limitations of clone detection: structural similarity does not necessarily imply derivation, so detected pairs may include convergent development, and a clone relationship does not tell which skill is older. Two properties of the benchmark matter for our purpose. For one thing, half of its 150 positive pairs are synthetic mutations rather than actual modifications. For another, its hard negatives are only same-category pairs whose similarity falls between 0.15 and 0.45, so very similar pairs written independently, like two skills generated from a single template, are not included. The families formed as connected components of the clone graph also form chains, and the largest one includes 61,651 skills. Near-duplicate detection has also reached developer tools: drskill gives an alert when two installed skills exceed a MinHash similarity threshold [1], and skillvariants groups near-clones of a skill found through GitHub code search [2]. Like SkillReuse, both tools find similar skills; neither decides whether one skill was copied from another.

2.4. Code clones and reuse on GitHub

Clone detection is an established area of research. Roy et al. [16] describe the usual types of clones and assess detection methods. SourcererCC performs token-based detection of near-miss clones at scale, for extremely large codebases [17]. Lopes et al. [15] found that file duplication is widespread on GitHub. Clone genealogies were introduced by Kim et al. [13], who track groups of clones across versions and distinguish changes applied consistently to all members of a group from inconsistent ones. These methods rely on lexical and syntactic information, which is reliable for source code. Skills, in contrast, are mostly written in natural language and are often generated from common templates, so surface similarity is a weaker signal of shared origin.

2.5. Summary

Table 1 summarises the approaches. Each earlier approach either needs an exact record that survives copying or measures similarity without deciding origin; ours decides origin from content and validates the decision against human labels.

3. Problem Statement

A developer adopts a skill by making a copy of the skill's folder in their own repository. The process of acquiring the skill does not involve any registry entry, manifest, or versioning, hence the adoption goes undocumented. Once the copied folder has been modified, tracing its source becomes a challenge. Three stakeholders bear the brunt of this practice. Adopters do not learn about fixes made to the source of a skill they adopted, while the agents that execute the skill continue to act on old and insecure instructions. A skill's author remains unaware of how far the skill has spread and what has been changed in it. A security reviewer who finds a dangerous skill cannot locate its modified copies.

Previous work follows the reuse path through a trace that remains unaltered during copying: the skill name registered in a registry [12], fork relationships [19], or an identical version shared with the source along with its change history [18]. In all cases, the reuse trail breaks when the copy is modified before it is committed, renamed, or translated, because nothing identical is left to identify reuse. Clone detection for skills [20] does not require any of these traces, but it compares how similar skills are rather than where they

Table 1: Summary of related work on tracing the reuse of agent skills.

Study	Data	How reuse is traced	Main finding	What it cannot do
Gao et al. [12]	18,463 registry and 23,199 GitHub skills	same YAML name, then text similarity	3,709 reuse links; 40.2% of unchanged copies miss later source changes	link renamed or heavily edited copies
Wu et al. [19]	forks of six popular skill repositories	GitHub fork relation	46 adaptation patterns; 18.6% add security-sensitive content	see copies pasted into unrelated repositories
Seddik [18]	GitSkills release, full git histories	identical versions joined by git edit history	2,193,119 adoptions; few repositories supply most copies	link copies edited before their first commit, renamed or translated
Zhu et al. [20]	137,470 marketplace skills; 300 labelled pairs	multi-channel similarity, logistic regression	F1 0.939; 66.8% of skills in a clone pair	separate copying from shared templates
This work	GitSkills release (1.59M skills); 400 + 120 labelled pairs	content classifier with boilerplate-aware features	F1 0.94; 0.99 precision of predicted links; family-level concentration	order derived pairs in time (Phase 4)

came from. It may consider two different skills built from the same template as clones, and its benchmark includes no highly similar skills written independently.

This gives rise to two problems. First, to our knowledge, no existing solution distinguishes edited copies from template siblings, which makes it impossible to tell reuse that exact matching cannot detect apart from shared scaffolding. Second, as a result, it is unclear what portion of reuse exact matching cannot detect. The problem also has practical implications: when a reviewer detects a harmful skill, the copy network can list its exact copies [18], but not the edited ones, which may still contain the harmful instructions.

4. Proposed Solution

We propose to identify derived variants based on the content of the skills alone, as a way of telling them apart from template siblings and measuring the reuse not captured by exact matching. Our approach has two steps.

The first step is a classifier that determines, for a particular pair of skills, whether one skill is a derived variant of the other, a template sibling of the other, or unrelated to it. The challenge is to distinguish between the three. Skills are typically short and often built from the same templates, so two independently written skills can be very similar, while a reworded or translated copy can share little text with its source. There is therefore no clear cut-off point for the degree of similarity, and training the classifier on labels obtained from such a cut-off would only teach it the cut-off point. We therefore developed our ground truth manually: team members independently labelled candidate pairs according to a written rubric, and we measured their agreement. Our sample covers all levels of similarity and deliberately contains highly similar template siblings, which SkillReuse-Bench excludes. In addition to lexical similarity, the classifier uses signals aimed at the template problem, including a similarity computed only over the shared lines that do not recur across many unrelated skills, together with front-matter, structural, and repository features. We compare it with exact hashing, MinHash-based and TF-IDF similarity, and a re-implementation of SkillReuse [20].

The second step applies the classifier to all candidate pairs of the entire GitSkills release [9], clusters skills into families, and calculates hidden reuse: what percentage of the derived pairs share neither a content hash nor a name, how many of them connect skills of different owners, and how accounting for them alters the view of how concentrated reuse is. Because these estimates rest on predictions, we check a random sample of predicted links by hand and report each estimate together with the measured precision.

This approach avoids the restrictions listed above. It requires no registry record, fork link, or shared version at all. Not needing fork links matters in practice: GitHub code search includes a fork only if it has more stars than the parent repository, so only three repositories in GitSkills are forks [10]. It distinguishes derivation from templating and validates this distinction on real pairs rather than synthetic mutations. It clusters skills into families with community detection, which limits how far a few bridging links can merge unrelated skills into one big family. Finally, it complements the copy network developed by Seddik [18]: that network covers exact copies along with their history, whereas we aim to find the edited copies it cannot connect. Both identify files by their git blob hash, which will allow us to match our links against that network directly in Phase 4.

5. Dataset Extraction

Source. The course offers GitSkills in the form of a single SQLite database, `agent_skills_release.db` [9], containing four tables: `artifacts`, one entry per file occurrence; `repos`, repository metadata; `artifact_siblings`, files bundled with each skill; and the `mining_runs`, metadata of data collection [11]. We rely on it on two levels. The *full release* (3,797,117 file occurrences of 1,877,981 distinct contents in 282,200 repositories owned by 195,841 accounts) serves as the population of all analyses. The *official sample* was trimmed down from the release by the authors of the dataset [10]: it contains the 13,000 distinct contents with the smallest content hashes, along with all rows related to them in the release, and it is the source of the pairs we labelled. The content hashes are effectively uniform, so the sample is a random sample of distinct skills; it is structured just like the release, and every labelled pair also occurs in the release.

Fields. Table 2 lists the fields we extract.

Table 2: GitSkills fields used in this study.

Field(s)	Use
`file_sha`, `repo_full_name`, `path`, `location_class`	occurrences, exact-copy counts
`filename`, `dedup_primary`, `frontmatter_valid`, `body_chars`	scope filtering
`content`, `name`, `description`	skill text (representatives)
`first_commit_at`	history coverage
`artifact_siblings.entry_name`	bundled-file similarity
owner (prefix of `repo_full_name`)	same-owner and cross-owner reuse

Table 3: Scope filters applied to the official sample and to the full release.

Filter	Sample	Full release
All distinct contents	13,000	1,877,981
Exact basename `SKILL.md`	12,763	1,842,600
Valid YAML front matter	11,197	1,610,957
Body $\geq$ 200 characters (dataset field)	11,010	1,586,606
Body $\geq$ 200 characters after our own parsing	11,009	1,586,429

Scope. Since the filename search used by GitSkills catches lowercase `skill.md` files as well as older files with similar names that predate the format, we retain only representatives whose basename is exactly `SKILL.md`. We also need valid front matter, since the name and description are used as inputs for our classifier, and a body of at least 200 characters, which filters out stub skills and skills stored as symbolic links (their bodies contain only a link target). Table 3 shows the effect of filtering on both levels; the ratio of contents that pass all filters is the same (85%), which is expected for a uniform sample.

The working set of the full release includes 1,586,429 skills, which are found in 3,341,056 files in 258,245 repositories owned by 181,349 accounts. For 389,594 of these skills, commit history is available, 577,916 bundle at least one other file, and 158,210 front-matter names are used by two or more distinct contents. The working set of the sample includes 11,009 skills.

Decomposition. Each representative file is decomposed into its normalised name, description, prose (body without fenced code blocks), code blocks, and headings. Additionally, we extract the repository context of each skill: the owner accounts of all of its occurrences, the name of the folder the skill is located in, and the names of the bundled files. At this stage we produce a new SQLite database with two tables, `skills` and `occurrences` (`extracted.db` for the sample, `full_skills.db` for the full release), which is the input of the following steps.

6. Study Design

Figure 1 gives an overview of our approach. It has five stages: the first two are automatic, the third is our manual analysis of the dataset, and the last two build the classifier and apply it.

Table 4: Similarity bands of the candidate pairs of the official sample and of the pairs drawn from them for labelling.

Band	Definition	Candidates	Sampled
B1	$J \geq 0.9$	426	60
B2	$0.7 \leq J < 0.9$	615	80
B3	$0.5 \leq J < 0.7$	1,278	80
B4	$0.3 \leq J < 0.5$	1,347	80
B5	same name, $J < 0.3$	1,889	50
B6	LSA neighbour, $J < 0.3$	1,901	30
B7	random pair (not a candidate)	–	20
B0	other LSH match, $J < 0.3$	628	0
	Total	8,084	400

Changes since Phase 2. Phase 3 extends the study in six ways, each answering a limitation of Phase 2. (i) The gold set doubles from 200 to 400 labelled pairs. (ii) Every analysis now runs on the full GitSkills release instead of the official sample. (iii) A blinded manual check measures the precision of the predicted links, so RQ3 is reported conditional on measured precision. (iv) A provenance check tests the labels against evidence that does not come from text similarity (Section 7.6). (v) The pipeline is made reproducible: package versions are pinned, the candidate pairs and the labelled sample are stored by content hash, threshold tuning no longer depends on floating-point details of a machine, fenced code is parsed as in CommonMark, and the labelling viewer is published with complete, untruncated skills. (vi) We report confidence intervals and significance tests, and we tried multilingual sentence embeddings to recover translated copies; they did not help (Section 7.4).

Stage 1: Data preparation. We extract the working set described in Section 5 and split each skill into its name, description, prose, code blocks, and headings, together with its repository context.

Stage 2: Candidate pair generation. Comparing every pair of the 1,586,429 skills would take more than a trillion comparisons, so we score only pairs proposed by inexpensive blockers. MinHash locality-sensitive hashing [5] over word 5-gram shingles of the body (128 permutations, 37 bands of 3 rows) proposes pairs whose estimated Jaccard similarity is around 0.3 or more, and same-name blocking proposes distinct contents that carry the same normalised front-matter name, capped at 45 pairs per name. On the sample we also used nearest neighbours in a latent semantic analysis (LSA) space [8] to catch reworded copies; in the labelled pairs, the pairs found only by LSA (band B6) contain no derived pair, so we drop this blocker at full scale. Large catalogs of near-identical skills fill single MinHash buckets with thousands of members; we pair each member of a bucket with more than 50 members with its 5 lowest-id members instead of with all of them, which keeps the bucket connected while the number of pairs stays linear (21,990 buckets were capped). On the full release the blockers propose 9,022,502 candidate pairs (Table 4 gives the bands of the sample, from which the labelled pairs were drawn).

Stage 3: Manual analysis. We followed three steps, explore, analyse, and synthesise, and added a fourth in Phase 3, validate.

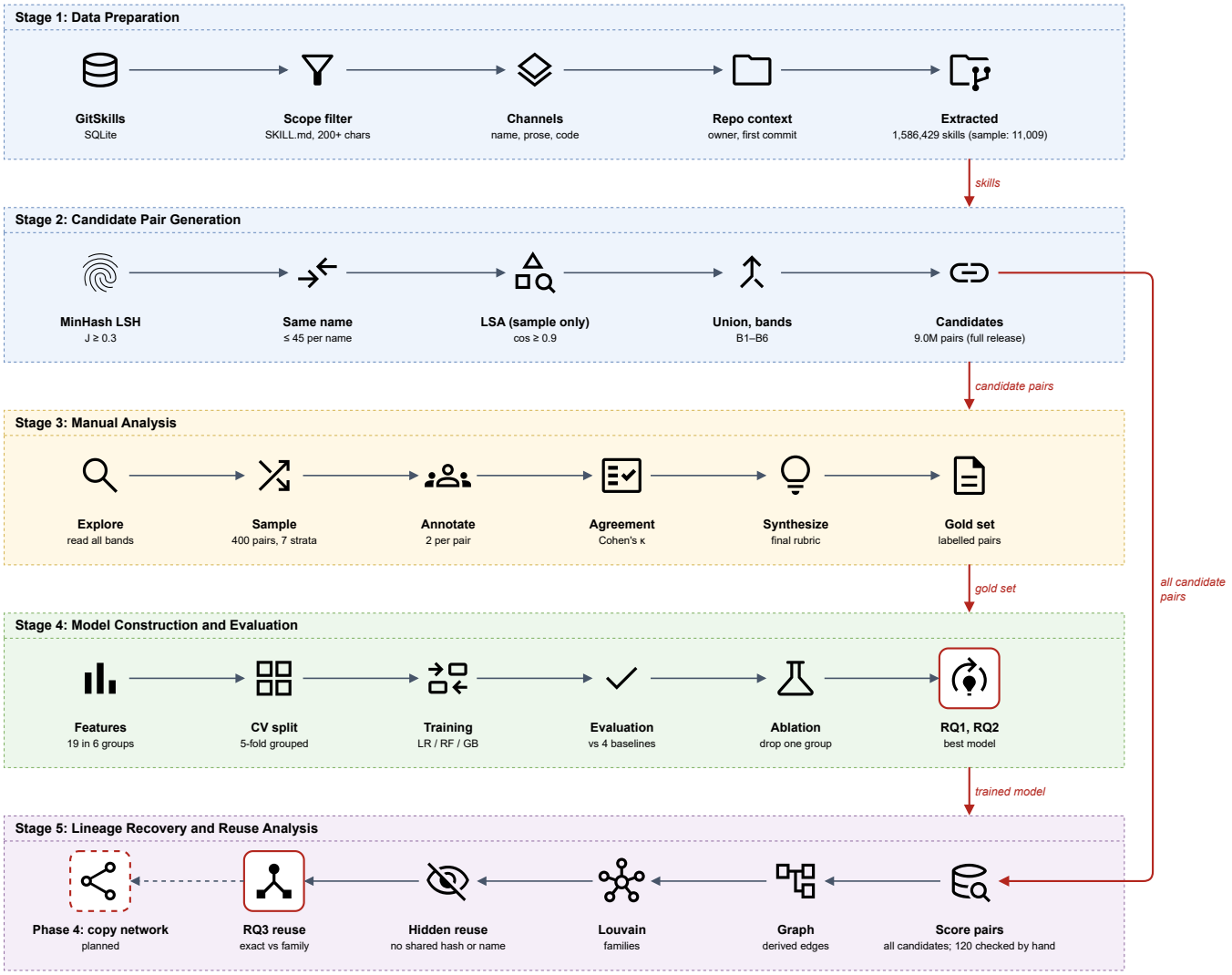


Figure 1: Approach overview. The shaded third stage is manual; outlined boxes answer our research questions, and the dashed box is planned for Phase 4. Stage 5 runs right to left, starting from the trained model and the full set of candidate pairs. The LSA blocker of Stage 2 is used on the official sample only.

Explore. Prior to annotation, we examined candidate pairs from all bands using a two-column side-by-side diff view along with selected skills to find out why particular skills resemble each other. Much of the similarity in the middle bands came from shared boilerplate rather than copying. For example, we found a collection of machine-generated skills named `u02156-...` that differed only in several words, skills that wrapped Rube MCP (Composio) boilerplate for multiple different applications, and also a series of German legal practice skills that featured the same checklist-like lines. On the other hand, there were several pairs with the same name and topic (such as two refactoring skills) which were created independently. These observations led to our three-class labelling scheme.

Analyse. Our stratified sample consisted of 400 pairs from all bands of Table 4, including 20 random pairs that no blocker had proposed, which serve as easy negatives. To keep the sample diverse, the same skill appears in at most two candidate pairs and the same name in at most four pairs; the random pairs were drawn without the skill limit, so one skill also appears in a third pair (P0044, P0145 and the random pair P0096). We randomised the left-right order of each pair and did not show similarity bands or scores to the annotators, so that neither could bias their labels. Pair identifiers were assigned after shuffling, so the first 200 pairs, which we labelled in Phase 2, are a random subset of the sample; in Phase 3 we labelled the remaining 200, for 400 pairs in total. Two of the team members independently annotated each pair according to criteria adapted from those Gao et al. used to validate their reuse links [12]. A pair was derived when the skills share distinctive content that is unlikely to be produced independently (identical or slightly modified text, as well as task-specific steps, examples, commands, paths, and configuration values; renaming and translating do not change this). A pair

was a template sibling if the only parts they shared were scaffolding text; otherwise the pair was unrelated, and the topic alone cannot be viewed as derivation. The annotators agreed on 388 of the 400 pairs, a Cohen's κ [7] of 0.95 over the three classes and 0.97 over the binarised data (i.e., derived vs. not derived). We resolved the 12 disagreements in discussion; all of them were borderline cases, such as a skill that reworks another one's steps without copying any line (P0121), or two skills that share a generated section schema but no content (P0185).[1]

Synthesise. From the notes we kept while labelling, we grouped the reasons why two skills look alike. Derived pairs arise from copy-and-edit, where a copy is trimmed or extended (P0000, P0046); from renamed copies (P0022, P0054, P0087); from translated copies (P0137); and from copies whose paths an installer rewrote for another agent tool (P0060, P0092). Template pairs arise from catalog generators that only fill slots (P0002, P0008), from integration catalogs that wrap many applications the same way (P0016, P0028), from domain series written by one author (P0036, P0068), and from long project preambles shared by different skills (P0003, P0058). Unrelated pairs often share a name or a topic but no text (P0038, P0084). Resolving the disagreements fixed three rules: a standard format of the domain, such as the sections of an architecture decision record, does not make two skills template siblings (P0142); a shared generated skeleton does, even without shared content (P0185); and a skill that reuses another one's distinctive steps is derived even when no line is copied verbatim (P0121), as is an installer copy with rewritten paths (P0169). Of the 75 pairs that at least one of us marked as uncertain, 24 come from band B5, where two skills share a name but little text, and 31 join two skills from the same owner. The resulting gold set contains 400 pairs: 82 derived, 240 template, and 78 unrelated.

Validate. Labels on a sample stratified by similarity show how well the classifier separates the classes, but not how precise its predictions are on the population. After applying the classifier to the full release, we therefore drew 80 predicted derivation links at random, together with 40 random candidate pairs that the classifier predicted as not derived, shuffled them, and labelled all 120 pairs with the same rubric and viewer, again two of us per pair. Annotators did not know which pairs the classifier had predicted as derived. They agreed on every pair ($\kappa = 1.00$); we attribute this to the pairs themselves, which are dominated by near-identical copies and catalog templates, while the gold set deliberately oversamples the harder bands.

Stage 4: Model construction and evaluation. For each pair we compute 19 features in six groups (Table 5). Corpus-level statistics come from the full release: a normalised line (lower case, list markers removed, whitespace collapsed, at least 20 characters containing a letter) is *boilerplate* when it appears in skills carrying at least 5 different front-matter names, which gives 676,696 boilerplate lines (2,476 on the sample alone); the TF-IDF, LSA and normalisation models are fitted once on about 50,600 skills of the release (50,000 chosen at random plus every skill of the gold set), which keeps memory use manageable. We train logistic regression, random forest and gradient boosting classifiers with balanced class weights and evaluate them with five-fold stratified group cross-validation, so that no skill appears in both a training and a test fold (Section 7.2). Section 6.1 opens this stage in detail.

[1]The labelling sheets and the gold set are part of our replication package: https://github.com/BhaveshMRA/RWPM.

Table 5: Pair features used by our classifier.

Group	Features
Lexical	Jaccard and containment of word 5-gram shingles; TF-IDF cosine of the bodies; normalised edit similarity of the first 5,000 characters
Boilerplate-aware	Jaccard and containment of distinctive lines; Jaccard of boilerplate lines; share of shared lines that are boilerplate
Metadata	names equal; name similarity; TF-IDF cosine of the descriptions
Code	Jaccard of identifier tokens in code blocks; both skills contain code
Structure	similarity of heading sequences; length ratio; heading-count ratio
Repository	a shared owner; same skill-folder name; Jaccard of bundled file names

Stage 5: Lineage recovery and reuse analysis. We train the best classifier from Stage 4 on all gold pairs and apply it to every candidate pair of the full release. Pairs predicted as derived become the edges of a derivation graph, which we split into families with Louvain community detection [4]. Because a few wrong edges can join large groups of skills, we build the families twice: from all predicted links, and, as a conservative variant, only from links between skills with the same front-matter name, which renamed copies cannot enter. We then measure hidden reuse and compare reuse counted by exact hash with reuse counted by family (RQ3). As a check on the labels themselves, we also record, for every labelled pair, evidence that does not come from text similarity: explicit attribution to a source, shared non-generic bundled files, and whether both skills have a recorded first commit.

Reproducibility. Package versions are pinned (`requirements-full.txt`, with the complete environments of both machines in `requirements-lock.txt` and `requirements-mac.txt`). The candidate pairs of the sample are frozen in a file keyed by content hash, and the labelled sample is stored the same way, so rerunning the pipeline cannot change either. Threshold tuning rounds scores before choosing a threshold, so floating-point differences between machines no longer change baseline decisions. Fenced code blocks are parsed as in CommonMark, and the labelling viewer, which now shows complete skills, is published with the labels. The full-release run streams the database in chunks; run on the official sample, it reproduces the features of the sample pipeline exactly.

6.1. White-Box View of the Classifier

Figure 1 shows the classifier only as the Training box of Stage 4. Figure 2 opens it and follows one candidate pair from its two `SKILL.md` files to the decision.

(1) Input. A candidate pair (a, b) proposed by the blockers of Stage 2.

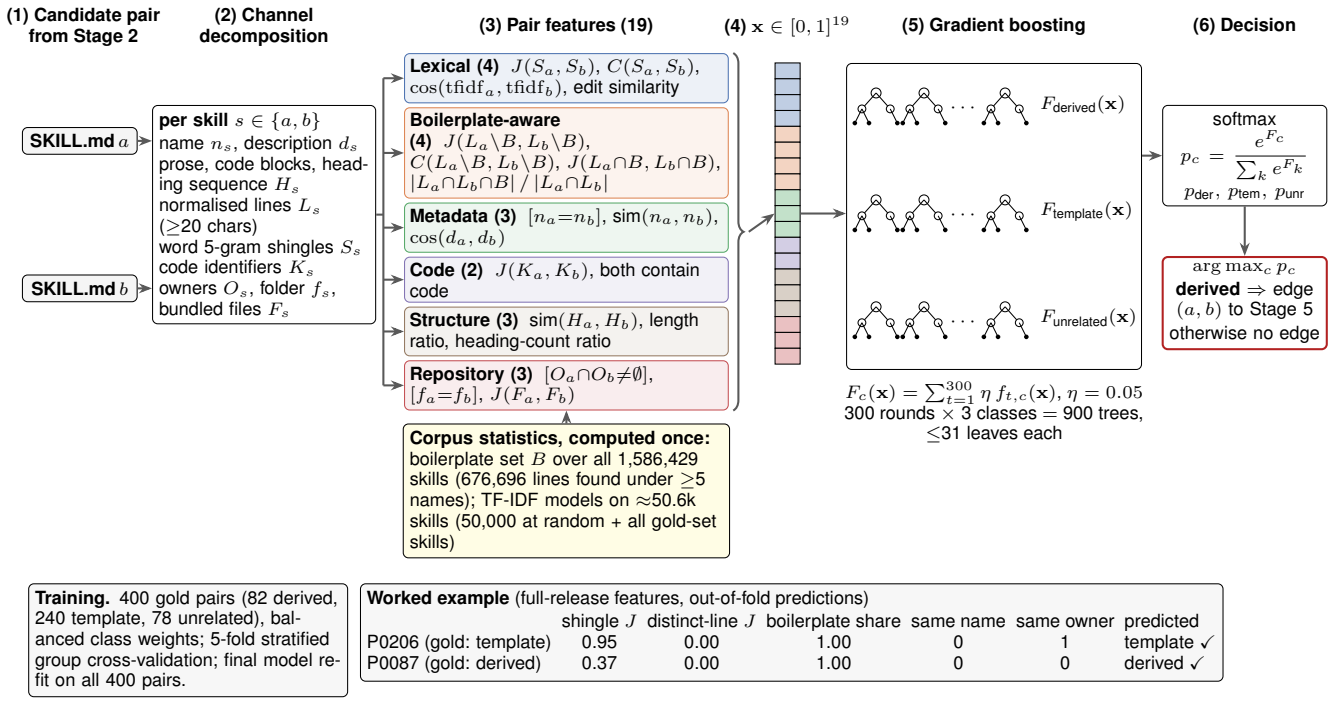


Figure 2: White-box view of the classifier (Stage 4 of Figure 1): how a candidate pair is decomposed into channels, turned into 19 features in six groups, and classified by gradient boosting; derived pairs become edges for Stage 5.

(2) Channel decomposition. Each file is split into its front-matter name n_s and description d_s, and its body into prose, code blocks and the sequence of headings H_s. From the body we derive the set of normalised lines L_s (lower case, list markers removed, whitespace collapsed, at least 20 characters containing a letter), the set of word 5-gram shingles S_s, and the set of identifiers K_s in code blocks (split at camel case and underscores). Repository context adds the owners O_s of all copies, the name of the skill's folder f_s, and the names of its bundled files F_s.

(3) Pair features. The two skills are compared channel by channel. Most features are a Jaccard similarity $J(X, Y) = |X \cap Y|/|X \cup Y|$, a containment $C(X, Y) = |X \cap Y|/\min(|X|, |Y|)$, a cosine of TF-IDF vectors, or an equality test. The boilerplate-aware group first removes the lines that belong to the boilerplate set B, so that $J(L_a \setminus B, L_b \setminus B)$ measures shared distinctive lines, and adds the share of shared lines that are boilerplate. Neither is computed per pair: B is counted once over all 1,586,429 skills of the release, the TF-IDF models are fitted once on about 50,600 of them, and both are reused for every pair.

(4) Feature vector. The 19 values form a vector $\mathbf{x} \in [0, 1]^{19}$; every feature is a similarity, a ratio or a binary indicator.

(5) Gradient boosting. The classifier runs 300 boosting rounds. In each round it fits one regression tree per class (at most 31 leaves, at least 20 training pairs per leaf, features binned into at most 255 values) to the gradient of the multinomial loss, so the model consists of 900 trees. The score of class c is $F_c(\mathbf{x}) = \sum_{t=1}^{300} \eta\, f_{t,c}(\mathbf{x})$ with learning rate $\eta = 0.05$, and training weights each class inversely to its frequency, so that 82 derived pairs weigh as much as 240 template pairs.

(6) Decision. A softmax turns the three scores into probabilities, and the most probable class is the prediction. A pair predicted as derived becomes an edge of the derivation graph of Stage 5; template and unrelated pairs add no edge.

Worked example. The bottom of Figure 2 shows two labelled pairs. At full scale neither shares a distinctive line: the lines of both pairs occur under five or more names, so all their shared lines are boilerplate. The boilerplate-aware features therefore cannot separate them, and plain similarity would rank them the wrong way round, since the template pair P0206 shares far more text (shingle Jaccard 0.95) than the derived pair P0087 (0.37). The model still classifies both correctly by combining the other groups; for example, the two skills of P0206 belong to one owner, which in the linear view of Figure 3 is the strongest single indicator of a template.

7. Experiments

7.1. Research Questions

Three research questions assess our solution.

RQ1 (Effectiveness). How accurately does our approach recognise derived skill variants, compared with exact hashing, similarity thresholds, and SkillReuse? This question tests whether derivation can be recognised from content alone, and how our design performs on the pairs that matter most: highly similar pairs that are not derived. We report precision, recall, F1, and the Matthews correlation

coefficient (MCC) [6] for the *derived* class, computed on pooled out-of-fold predictions, together with the standard deviation of F1 across folds and 95% bootstrap confidence intervals. We also report the false-positive rate on high-similarity pairs (non-derived pairs in bands B1–B3), precision and recall per band, and, for our approach, a three-class confusion matrix and macro-F1.

RQ2 (Contributing signals). Which signals contribute most to separating derived skill variants from template siblings and unrelated skills? We remove one feature group at a time, retrain the best model, and measure the change in F1 and in the high-similarity false-positive rate. We also train a model on lexical features alone.

RQ3 (Hidden reuse). How much skill reuse is hidden from exact and name matching, and how does counting it change the picture of reuse? This question measures what hash-, name-, and version-based approaches leave out. Among the labelled derived pairs, we measure the share whose front-matter names differ; since the two skills of every pair also have different content hashes, these pairs are invisible to both exact and name matching. Among all derivations predicted by the best model, we measure the share with different names and the share that crosses owners. We then compare two units of reuse, exact hash and derivation family, by the number of units, the inflation ratio (distinct contents per family), the size of the largest family, the Gini coefficient of file occurrences per unit, and the share of units spanning two or more owners. We leave the comparison with the copy network of Seddik [18] to Phase 4.

7.2. Experimental Setup

RQ1 and RQ2 use the 400 gold pairs; RQ3 uses the predictions on the full release and the manually checked sample of them. We use five-fold stratified group cross-validation; groups are formed by connected components of the labelled-pair graph, so no skill contributes to both training and testing. We compare four baselines with our approach.

Normalised-text hash predicts *derived* only when the two bodies are identical after lower-casing and collapsing whitespace, which is what exact hashing sees once trivial formatting is ignored. *Shingle Jaccard (MinHash)* thresholds the exact Jaccard similarity of word 5-gram shingles, the quantity MinHash estimates. *TF-IDF cosine* thresholds the cosine similarity of TF-IDF vectors of the bodies. For both, the threshold that maximises F1 is chosen on the training folds only. *SkillReuse-style* is our re-implementation of the eight features of Zhu et al. [20]: a flat TF-IDF similarity, LSA cosine similarities for the YAML, prose, and code channels (set to zero when a channel is missing), their three pairwise products, and a structural similarity over eight document statistics, combined by a class-balanced logistic regression whose threshold is again tuned on the training folds. It is trained on our gold set, so it is compared with our approach on equal terms; it is not the authors' released implementation.

Our approach uses the 19 features of Table 5 with three classifiers from scikit-learn: logistic regression on standardised features, a random forest with 500 trees, and histogram-based gradient boosting, all with balanced class weights and no threshold tuning. The classifiers are trained on the three classes when the gold set has at least 15 template pairs, and on derived versus not derived otherwise. We report 95% bootstrap confidence intervals (2,000 resamples of the labelled pairs) and compare our best model with the strongest baseline using McNemar's exact test. On the sample we also tried adding multilingual sentence embeddings (`paraphrase-multilingual-MiniLM-L12-v2`) as a sixth feature group, aimed at translated and reworded copies (Section 7.4).

7.3. RQ1: Effectiveness

Table 6 compares all methods. The best configuration of our approach, gradient boosting, reaches a precision of 0.97, a recall of 0.90, and an F1 of 0.94 (95% CI 0.89–0.97; MCC 0.92); random forest is almost as good (F1 0.94), while logistic regression drops to 0.87 with full-corpus features, presumably because some feature distributions become very skewed. The strongest baseline, the SkillReuse-style detector, reaches an F1 of 0.74 (0.66–0.81). The difference in F1 is 0.13-0.27 (95% CI), and the two methods disagree in a way that is very unlikely by chance: ours is right on 36 pairs that the baseline gets wrong, the baseline on 3 (McNemar's test, $p < 0.001$). Among the 159 highly similar pairs that annotators judged not derived, our approach labels none as derived, the SkillReuse-style detector 10%, and MinHash similarity 33%. Across the three classes our approach reaches a macro-F1 of 0.95 (Table 8).

The confusion matrix (Table 8) shows where the model succeeds and where it fails. It recognises template siblings almost perfectly: 238 of the 240 template pairs are classified correctly and none is called derived, so the confusion behind most baseline false positives does not occur. This is where the design pays off: a template pair shares mostly boilerplate and, typically, an owner, while its names and descriptions differ in the substituted slots. The errors lie elsewhere: eight derived pairs are missed (four called template, four unrelated), two unrelated pairs are called derived, and two template and two unrelated pairs are confused with each other, which affects no derivation link.

Of the 96 false positives of the three similarity baselines, 90 are template pairs, and of the 19 derived pairs that all three miss, our approach finds 13; examples are P0054, which renames the grill-me skill and rewrites its rules around one copied sentence, and P0169, an installer copy of the gsd-manager skill whose paths were rewritten. On the derived class our model makes ten errors (eight missed copies and two false alarms), in three groups. First, four copies share no text with their source: P0121 reworks another skill's steps, P0137 and P0268 are Chinese adaptations, and P0335 keeps only the description; with no shared text, the model calls them unrelated. Second, four copies of two very popular skills, Matt Pocock's improve-codebase-architecture (P0053, P0123, P0297) and Superpowers' verification-before-completion (P0348), three of them renamed, are called templates: at full scale their lines occur under five or more names and therefore count as boilerplate. Third, the two false positives, P0226 and P0270, are unrelated skills that share a name and a topic.

> **Answer to RQ1:** Our approach identifies derived variants with an F1 of 0.94 (0.89-0.97), significantly better than the strongest baseline (0.74), and labels no highly similar non-derived pair as derived. Its errors are copies without shared text, copies of very popular skills, and same-name skills on the same topic.

Table 6: RQ1: detection of derived variants (pooled out-of-fold predictions; F1 ± standard deviation across folds). The last column is the share of non-derived pairs in bands B1–B3 that a method labels as derived (lower is better).

Method	P	R	F1	MCC	FP rate (hi-sim)
Normalised-text hash	0.71	0.06	0.11±0.07	0.17	0.01
Shingle Jaccard (MinHash), tuned threshold	0.46	0.54	0.49±0.13	0.35	0.33
TF-IDF cosine, tuned threshold	0.59	0.37	0.45±0.08	0.36	0.13
SkillReuse-style (re-implemented)	0.73	0.76	0.74±0.05	0.67	0.10
Ours (logistic regression)	0.86	0.88	0.87±0.05	0.83	0.06
Ours (random forest)	0.99	0.89	0.94±0.05	0.92	0.00
Ours (gradient boosting)	0.97	0.90	0.94±0.06	0.92	0.00

Table 7: RQ1 by similarity band: precision (P) and recall (R) of the strongest baseline and of our best model.

				Best baseline		Ours	
Band	Definition	Pairs	Derived	P	R	P	R
B1	$J \geq 0.9$	60	33	0.92	1.00	1.00	1.00
B2	$0.7 \leq J < 0.9$	80	21	0.65	0.81	1.00	1.00
B3	$0.5 \leq J < 0.7$	80	7	0.60	0.86	1.00	0.71
B4	$0.3 \leq J < 0.5$	80	13	0.86	0.46	1.00	0.85
B5	same name, $J < 0.3$	50	8	0.00	0.00	0.67	0.50
B6	LSA neighbour, $J < 0.3$	30	0	–	–	–	–
B7	random pair	20	0	0.00	–	–	–

Table 8: RQ1: confusion matrix of our best model (out-of-fold).

True ↓ / Predicted →	*derived*	*template*	*unrelated*
derived	74	4	4
template	0	238	2
unrelated	2	2	74

Table 9: RQ2: ablation of feature groups for our best model.

Configuration	F1	ΔF1	FP rate (hi-sim)
All features	0.94	–	0.00
without lexical	0.90	-0.04	0.03
without boilerplate	0.91	-0.03	0.01
without metadata	0.88	-0.05	0.01
without code	0.91	-0.03	0.01
without structure	0.92	-0.02	0.01
without repository	0.91	-0.03	0.01
lexical only	0.59	-0.34	0.25

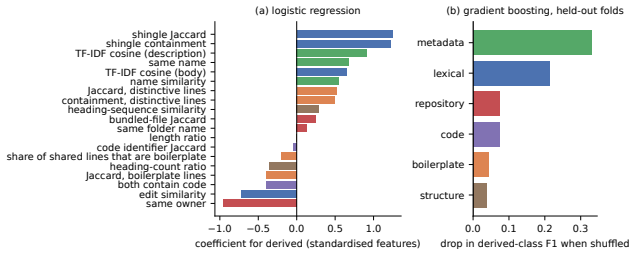


Figure 3: What the trained model relies on (complementing the white-box view of Figure 2): (a) logistic-regression coefficients for the derived class on standardised features; (b) drop in derived-class F1 of the best model on held-out folds when the features of one group are shuffled together.

7.4. RQ2: Contributing Signals

Table 9 reports the ablation. Lexical similarity alone is far from enough: a lexical-only model drops to an F1 of 0.59 and labels a quarter of the highly similar non-derived pairs as derived. No single other group is indispensable; removing any one costs between 0.02 and 0.05 in F1, most for metadata (0.05). Figure 3 shows what the best model relies on. On held-out folds, shuffling the metadata features costs most (0.33 in F1), followed by lexical (0.21), repository (0.08) and code (0.07) features; boilerplate-aware features cost only 0.05. On the sample alone, boilerplate-aware features had been the most important group (0.39). With boilerplate counted over the full release, the lines of popular skills that are copied under many names become boilerplate too, so these features separate templates from copies less sharply and the model leans on names and descriptions instead. Adding multilingual sentence embeddings did not help: on the sample, F1 changed from 0.944 to 0.938 (McNemar's test, $p = 1.0$), the embeddings fixed two pairs and broke three, and the translated copies they were meant to catch stayed undetected, because embedding similarity measures topic as much as origin.

> **Answer to RQ2:** Signals beyond lexical similarity are essential (F1 0.59 with lexical features alone). At full scale, names and descriptions contribute most and boilerplate-aware features least, because boilerplate

Table 10: Manual check of predicted links on the full release (precision with 95% Wilson interval).

Links	Checked	Derived	Precision
All predicted links	80	79	0.99 [0.93, 1.00]
same name	60	60	1.00 [0.94, 1.00]
different names	20	19	0.95 [0.76, 0.99]
Decoys (predicted not derived)	40	0	–

Table 11: RQ3: reuse counted by exact hash and by derivation family on the full release.

Metric	Exact hash	Family all links	Family same name
Units	1,586,429	1,117,691	1,169,540
Gini of occurrences per unit	0.49	0.63	0.61
Occurrences in top 1% of units	27.0%	42.0%	40.1%
Units spanning ≥2 owners	13.4%	16.2%	16.0%
Occurrences in multi-owner units	52.3%	65.7%	64.5%

counted over 1.59 million skills also absorbs the lines of popular copied skills.

7.5. RQ3: Hidden Reuse

Applied to the 9,022,502 candidate pairs of the full release, the best model predicts 4,058,198 derivation links. Of 80 of these links drawn at random, our manual check confirms 79, a precision of 0.99 (Table 10): all 60 sampled links between same-name skills and 19 of the 20 links between skills with different names were derived, and none of the 40 decoys was. Of the predicted links, 18.0% join skills with different names, which neither exact nor name matching can connect (24.4% of the labelled derived pairs); with the measured precision of such links (0.95, 95% CI 0.76-0.99), about one in six predicted derivations is hidden reuse. 92.4% of the predicted links cross owners. The one wrong link in the sample, V100, joins two skills of a Chinese fund-analysis catalog that were generated from the same scaffold (identical headings, input types, quality requirements, run commands, and output skeleton) for different tasks. It is a template pair, which the model took for a copy, presumably because this catalog's scaffold recurs under too few names for its lines to count as boilerplate. None of the 40 decoys was derived, so the sample contains no missed link; Section 7.3 discusses the derivations the model misses.

Table 11 compares reuse counted by exact hash with reuse counted by family, for families built from all links and from same-name links only.

Both variants tell the same story: counting families instead of hashes raises the Gini coefficient from 0.49 to 0.61–0.63 and the share of all files held by the top 1% of units from 27% to 40–42%. Popular skills exist in hundreds of edited versions that exact hashing counts as unrelated units: the frontend-design family contains 966 distinct contents with 4,564 files, and skill-creator, systematic-debugging, writing-plans and brainstorming follow. The two variants differ in their largest groups. With all links, the largest connected component has 4,564 skills; it and the next four join different skills of one ecosystem (the gstack commands, frontend design skills, the Composio integration catalog, skill-creator and related skills, and the spec-kit commands) through a small number of links between different skills. Such links are rare among all predictions, so the sample of 80 contains only one wrong link, but a few of them suffice to merge families. With same-name links only, the largest family has 966 skills, each of the ten largest families is a single skill, and the concentration figures barely change. The copy network of Seddik [18] finds that a few repositories supply almost all exact copies; our figures show, from the side of the copied skills, that reuse is also concentrated on a few skills, and more strongly once edited copies are counted.

Table 12: Provenance evidence per gold-set class.

Evidence	derived (n = 82)	template (n = 240)	unrelated (n = 78)
Explicit attribution to a source	8.5%	0.0%	0.0%
Names the other skill's repository or account	0.0%	0.0%	2.6%
Shared non-generic bundled file	22.0%	23.8%	0.0%
Both have a recorded first commit	4.9%	5.0%	6.4%

> **Answer to RQ3:** About one in six predicted derivations joins skills with different names and is therefore hidden from exact and name matching; most derivations cross owners. Counted by family, reuse is markedly more concentrated than counted by exact hash (Gini 0.61–0.63 instead of 0.49), with or without the links between differently named skills. These figures are conditional on the predicted links, whose precision we measured at 0.99 (0.93-1.00).

7.6. Validity of the Labels

Our labels judge content against a rubric; they do not observe copying. Table 12 checks them against evidence that does not come from text similarity. An explicit attribution to a source, such as "Credits: original skill by @blader" or "adapted from mattpocock/skills", appears in 8.5% of derived pairs and in none of the template or unrelated pairs; one attribution confirms P0137, a translated copy that our model misses. Such statements are rare, so they confirm labels rather than replace them. Shared bundled files do not separate the classes, because template catalogs ship the same scripts with every skill, and both skills of a pair have a recorded first commit in only about 5% of pairs, too few to order derived pairs in time. In the validation sample, annotators also found an edited copy that appends a reverse-shell section to an otherwise verbatim ClickHouse skill (V103, from a benchmark of malicious skills): exactly the kind of edited relative that exact matching cannot connect to its source.

8. Threats to Validity

Conclusion validity. Our conclusions rest on limited samples: the gold set has 400 pairs, only 82 of them derived, so one error moves recall by about a point, and the precision of links between differently named skills rests on 20 checked links (95% CI 0.76-0.99). We therefore report bootstrap confidence intervals and compare our model with the strongest baseline using McNemar's exact test instead of comparing point estimates. The measurement itself, human labels, is checked by double labelling: agreement is 0.95 on the

stratified gold set and 1.00 on the validation sample, whose pairs are mostly clear-cut and therefore test the rubric less.

Internal validity. We chose the best of three classifiers by cross-validated F1, which slightly favours our approach; the random forest performs almost as well, so the conclusions do not hinge on that choice. Some features may capture correlates rather than causes of derivation: the same-owner feature can act as a shortcut for "template", because many catalogs belong to a single owner, and at full scale the boilerplate threshold of five names also covers the lines of popular skills copied under many names, which shifts the model towards metadata and accounts for four of its ten errors on the derived class. Families depend on a few links between different skills; we bound this effect with the same-name variant. At full scale we dropped the LSA blocker; in the labelled sample, the pairs it alone proposed contain no derived pair.

Construct validity. The labels come from four graduate students applying a written rubric, not from the authors of the skills. A derived label means that two skills share distinctive content that is unlikely to be written twice independently; it does not prove historical copying, and explicit attribution confirms only a small share of derived pairs (Section 7.6). Hidden reuse is measured by front-matter names, so a renamed copy counts as hidden even when its folder name reveals the source. Our indicators are chosen for the task: F1 and MCC on the derived class, the false-positive rate on highly similar non-derived pairs, which measures the template problem directly, and the precision of predicted links; recall on the full release is observed only through the decoys of the validation sample.

External validity. We study one dataset: public GitHub repositories, default branches only, as indexed by GitHub code search, collected in July 2026, early in the adoption of the format [11]. Skills in registries and marketplaces such as skills.sh or SkillsMP, and in private repositories, may be reused differently. The data are real repositories, not synthetic mutations, and the 1.59 million skills come from 258,245 repositories of very different kinds; the labelled pairs come from a uniform sample of the release, although stratification oversamples similar pairs. Candidates come from same-name blocking and MinHash, so heavily reworded copies under a new name may never be scored, while classification errors can move the hidden-reuse share either way; the share is therefore an estimate for the predicted links, not a bound. Our SkillReuse baseline is re-implemented from the paper and trained on our labels; its numbers are not comparable to those of Zhu et al. [20].

9. Conclusion

Pairs of agent skills that our rubric labels as derived can be distinguished based on their content alone and are clearly separated from template siblings: on 400 labelled pairs our classifier achieves an F1 score of 0.94, compared to 0.74 for a detector designed in the manner of SkillReuse, and it does not flag any of the highly similar template or unrelated pairs as derived. Of 80 predictions drawn at random from the full GitSkills release, 79 were confirmed manually. Analysing these predictions shows that reuse is much more concentrated than the number of exact copies suggests, and that about one in six predicted derivations cannot be found by exact or name matching.

10. Future Work

In Phase 4 we will (i) scale the boilerplate threshold with the size of the corpus, so that the lines of popular copied skills remain distinctive, and limit the links that pass through bucket anchors; (ii) label more cross-language and heavily modified copies, which account for half of the missed derivations; (iii) compare our derived links with the copy network of Seddik [18] using the git blob hashes both datasets share, to see how many of our links that network misses and to order derived pairs in time; and (iv) release the classifier as a tool that lists the edited copies of a given skill.

Data Availability

Our replication package contains the code of every analysis step, from data extraction to the figures in this paper, the labelling sheets, gold set and validation labels, and all generated results; its README gives the run order for the official sample and for the full release. It is available at https://github.com/BhaveshMRA/RWPM/tree/phase3.

Acknowledgments

We used an AI assistant (Claude Opus 5.5, Anthropic) for brainstorming the research direction; for searching the literature, which led us to the Skill Constellations study [18]; for help in debugging the analysis scripts and assisting with the approach overview figure; for a review of labels and notes after the annotators had finished labelling, which changed some confidence values and notes but no labels; and for suggestions during the adjudication of disagreements, where the final decisions were ours. All labels were assigned by the authors, and the authors take full responsibility for the content of this paper.

References

[1] 2026. drskill: a checker for the skills and MCP servers an AI coding agent loads. Python package, https://pypi.org/project/drskill-core/.

[2] 2026. skillvariants: studying how Agent Skills have been adapted across GitHub. Python package, https://pypi.org/project/skillvariants/.

[3] Agent Skills. 2025. Agent Skills Specification. https://agentskills.io.

[4] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast Unfolding of Communities in Large Networks. *Journal of Statistical Mechanics: Theory and Experiment* 2008, 10 (2008), P10008. https://doi.org/10.1088/1742-5468/2008/10/P10008

[5] Andrei Z. Broder. 1997. On the Resemblance and Containment of Documents. In *Proceedings of Compression and Complexity of Sequences*. 21–29. https://doi.org/10.1109/SEQUEN.1997.666900

[6] Davide Chicco and Giuseppe Jurman. 2020. The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation. *BMC Genomics* 21 (2020), 6. https://doi.org/10.1186/s12864-019-6413-7

[7] Jacob Cohen. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46. https://doi.org/10.1177/001316446002000104

[8] Scott Deerwester, Susan T. Dumais, George W. Furnas, Thomas K. Landauer, and Richard Harshman. 1990. Indexing by Latent Semantic Analysis. *Journal of the American Society for Information Science* 41, 6 (1990), 391–407.

[9] Giuseppe Destefanis, Daniel Graziotin, Matteo Vaccargiu, and Marco Ortu. 2026. GitSkills: A Dataset of Agent Skills on GitHub, release v1.0.0 (agent_skills_release.db). https://doi.org/10.5281/zenodo.21875637

[10] Giuseppe Destefanis, Daniel Graziotin, Matteo Vaccargiu, and Marco Ortu. 2026. GitSkills: Sample for the MSR 2027 Mining Challenge Proposal. https://github.com/giuseppedestefanis/gitskills-sample.

[11] Giuseppe Destefanis, Daniel Graziotin, Matteo Vaccargiu, and Marco Ortu. 2027. GitSkills: A Dataset of Agent Skills on GitHub. In *Proceedings of the 24th International Conference on Mining Software Repositories (MSR '27)*. ACM. To appear. arXiv:2608.10906.

[12] Haoyu Gao, Jai Lal Lulla, Hong Yi Lin, Sebastian Baltes, Christoph Treude, and Mansooreh Zahedi. 2026. From Registry to Repository: How AI Agent Skills Are Written, Adapted, and Maintained. arXiv:2607.00911.

[13] Miryung Kim, Vibha Sazawal, David Notkin, and Gail C. Murphy. 2005. An Empirical Study of Code Clone Genealogies. In *Proceedings of the 10th European Software Engineering Conference held jointly with the 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE)*. https://doi.org/10.1145/1095430.1081737

[14] Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. 2026. Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale. arXiv:2601.10338.

[15] Cristina V. Lopes, Petr Maj, Pedro Martins, Vaibhav Saini, Di Yang, Jakub Zitny, Hitesh Sajnani, and Jan Vitek. 2017. DéjàVu: A Map of Code Duplicates on GitHub. *Proceedings of the ACM on Programming Languages* 1, OOPSLA, Article 84 (2017). https://doi.org/10.1145/3133908

[16] Chanchal K. Roy, James R. Cordy, and Rainer Koschke. 2009. Comparison and Evaluation of Code Clone Detection Techniques and Tools: A Qualitative Approach. *Science of Computer Programming* 74, 7 (2009), 470–495. https://doi.org/10.1016/j.scico.2009.02.007

[17] Hitesh Sajnani, Vaibhav Saini, Jeffrey Svajlenko, Chanchal K. Roy, and Cristina V. Lopes. 2016. SourcererCC: Scaling Code Clone Detection to Big-Code. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. 1157–1168. https://doi.org/10.1145/2884781.2884877

[18] Fahd Seddik. 2026. Skill Constellations: Tracing the Supply Chain of Agent Skills on GitHub. arXiv:2610.11169.

[19] Xinjian Wu, Jingzhi Gong, Gunel Jahangirova, Zhenpeng Chen, and Jie M. Zhang. 2026. An Empirical Study of Downstream Adaptation for Agent Skills. arXiv:2607.03238.

[20] Jiaying Zhu, Lyuye Zhang, Wenbo Guo, and Yang Liu. 2026. Latent Reuse in Agent Skills: Multi-modal Clone Detection at Ecosystem Scale. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*. https://doi.org/10.1145/3832783.3837468